

1. 函数对象

函数对象 (Function Object) 又称函数对象类、仿函数、高阶函数等, 是指那些可以被传入其他函数或从其他函数返回的函数。函数对象实际上是一个重载了 `operator()` 操作符的类, 由于重载了这个操作符的类有和普通函数类似的使用方法, 所以称其为函数对象。

- `()`

举例

```
void fun()
{
    cout << "hello world" << endl;
}
int main()
{
    fun();
}
```

- 功能

让一个对象可以像函数一样去使用

- 特点
 - 对参数的个数, 参数类型, 返回值类型没有限制, 看具体要实现的功能
- 实现方式

只能以成员函数方式

- 成员函数方式

`o(形参表)` 的表达式可以被编译器翻译成 `o.operator()(形参表)`。

```
#include<iostream>
using namespace std;

class Func
{
public:
    double operator()(double d)
    {
        return d * d;
    }
};

int main()
{
    Func func;
    //func.operator()(3.14)
    cout << func(3.14) << endl;
    return 0;
}
```

函数对象的使用非常灵活，可以像普通函数那样调用，有参数，也可以有返回值。此外，函数对象还可以有自己的状态，这使其超出普通函数的概念。它们可以作为参数传递给其他函数。

2. lambda匿名函数

有些时候，对于一些比较简单且使用次数少的函数，我们没必要再特地去定义一个函数去实现该功能。例如，求一个数的平方

```
double square(int x)
{
    return x * x;
}
```

此时我们可以使用 lambda匿名函数来完成上述操作

2.1 lambda函数的定义

lambda被用来表示一种匿名函数.所谓匿名函数,简单地理解就是没有名称的函数.又常被称为lambda函数或者lambda表达式。

lambda表达式创建一个匿名函数对象，通常被称为lambda函数或lambda闭包。当你定义一个lambda表达式时，你实际上创建了一个可以像函数一样调用的对象。这个对象有一个调用运算符（operator()），这使得它可以像函数那样被调用。

因此，当你写下一个lambda表达式并赋值给一个变量时，这个变量的类型是一个函数对象，而不是一个返回值。lambda表达式本身没有“返回值”的概念，因为它不是一个函数调用，而是一个对象定义。

2.2 lambda函数的语法

- 定义

```
[外部变量访问方式说明符] (参数) mutable noexcept/throw() -> 返回值类型
{
    函数体;
};
```

- 各部分的含义分别为

- [外部变量访问方式说明符]

[]方括号用于向编译器表明当前是一个lambda表达式,其不能被省略。在方括号内部,可以注明当前lambda函数的函数体中可以使用哪些“外部变量”。所谓外部变量,指的是和当前lambda表达式位于同一作用域内的所有局部变量。

比如,如下就定义了一个最简单的lambda匿名函数:

```
auto d = [] {};// 一个空的lambda表达式
```

- (参数)

和普通函数的定义一样,lambda匿名函数也可以接收外部传递的多个参数.和普通函数不同的是,如果不需要传递参数,可以连同()小括号一起省略

```

auto c = []/*()*/{
    int a = 1;
    int b = 2;
    return a / b;
};
cout << c() << endl;

auto square = [](int x)/* -> int*/ {
    return x;
};
cout << square(1) << endl;

```

- mutable

此关键字可以省略,如果使用则之前的()小括号将不能省略(参数个数可以为0)。默认情况下,对于以值传递方式引入的外部变量,不允许在lambda表达式内部修改它们的值(可以理解为这部分变量都是const常量)。而如果想修改它们,就必须使用mutable关键字。

注意:对于以值传递方式引入的外部变量,lambda表达式修改的是拷贝的那一份,并不会修改真正的外部变量

[外部变量]的定义方式

外部变量格式 功能

- [] 空方括号表示当前lambda匿名函数中不导入任何外部变量.
- [=] 只有一个=等号,表示以值传递的方式导入所有外部变量.

```

auto d = [=](int i) mutable {
    int x = 3;
    int y = 4;
    a = 10; //lambda表达式修改的是拷贝的那一份,并不会修改真正的外部变量
    return a + b + i + x + y;
};
//默认是值传递
auto c = [=](int i) /*-> float*/ {
    int x = 3;
    int y = 4;
    //a = 10; //error 以值传递方式引入的外部变量,不允许在lambda表达式内部修改它们的值
    return a + b + i + x + y;
};

cout << c(5) << endl;
cout << d(5) << endl;

```

- [&] 只有一个&符号,表示以引用传递的方式导入所有外部变量.

```
int a = 1;
int b = 2;

auto c = [&](int i) /*-> float*/ {
    int x = 3;
    int y = 4;
    a = 10; // 引用传递可以修改
    return a + b + i + x + y;
};

cout << c(5) << endl;
```

- `[val1, val2, ...]` 表示以值传递的方式导入 `val1, val2` 等指定的外部变量,
- 同时多个变量之间没有先后次序

```
int a = 1;
int b = 2;
int c = 3;

auto d = [a, b](int i) /*-> float*/ {
    int x = 3;
    int y = 4;
    // return a + b + c + i + x + y; // error c 没有传入, 不能使用
    return a + b + i + x + y;
};

cout << d(5) << endl;
```

- `[&val1, &val2, ...]` 表示以引用传递的方式导入 `val1, val2` 等指定的外部变量,
- 多个变量之间没有前后次序

```
int a = 1;
int b = 2;
int c = 3;

auto d = [&b, &a](int i) /*-> float*/ {
    int x = 3;
    int y = 4;
    // return a + b + c + i + x + y; // error c 没有传入, 不能使用
    return a + b + i + x + y;
};

cout << d(5) << endl;
```

- `[val1, &val2, ...]` 以上2种方式还可以混合使用, 变量之间没有前后次序。


```

int a = 1;
int b = 2;
int c = 3;

auto d = [&b, a](int i) /*-> float*/ {
    int x = 3;
    int y = 4;
    //return a + b + c + i + x + y; //error c没有传入, 不能使用
    return a + b + i + x + y;
};

cout << d(5) << endl;

```

- [=,&val1,...] 表示除val1以引用传递的方式导入外,其它外部变量都以值传递的方式导入

```

int a = 1;
int b = 2;
int c = 3;

auto d = [=, &b](int i) /*-> float*/ {
    int x = 3;
    int y = 4;
    return a + b + i + x + y;
};

```

- [this] 表示以值传递的方式导入当前的this指针。

```

#include <iostream>

class MyClass {
public:
    void fun()
    {
        // 使用 [this] 捕获列表访问和修改成员变量和成员函数
        auto lambda = [this]()
        {
            // 访问局部变量
            std::cout << "Local variable: " << this->memberVar << std::endl;
        };
        lambda();
    }
private:
    int memberVar = 5;
};

int main() {
    MyClass obj;
    obj.fun();

    return 0;
}

```

- noexcept/throw()

可以省略,如果使用,则之前的()小括号将不能省略(参数个数可以为0)。默认情况下,lambda函数的函数体内可以抛出任何类型的异常.而标注**noexcept**关键字,则表示函数体内不会抛出任何异常.使用**throw()**可以指定**lambda函数内部可以抛出的异常类型**。如果lambda函数标有**noexcept**而函数体内抛出了异常,又或者使用**throw()**限定了异常类型而函数体内抛出了非指定类型的异常,这些异常无法使用try-catch捕获,会导致程序执行失败。

```
#include <iostream>
using namespace std;

int main()
{
    auto d = [](/*int x, int y*/)throw()/*noexcept*/
    {
        int x = 1;
        int y = 0;
        if (y == 0)
        {
            throw(-1);
            //throw string("除数不能为0");
            //throw(1.1);
        }
        return x / y;
    };
    try
    {
        cout << d(/*1, 0*/) << endl;;
    }
    catch (int n)
    {
        cout << "除数不能为0" << endl;
    }
    catch (const string& str)
    {
        cout << str << endl;
    }

    auto e = [](/*int x, int y*/)noexcept
    {
        int x = 1;
        int y = 2;
        return x / y;
    };
    cout << e() << endl;

    return 0;
}
```

◦ 返回值类型

指明lambda匿名函数的返回值类型.大部分情况下可以直接省略->返回值 类型。

```
#include<iostream>
using namespace std;

class A
{

```

```

public:
    double m_data;
};

int main()
{
    A a1;
    auto c = [&]() /*-> A*/
    {
        double a = 3.0;
        double b = 2.0;
        double res = a / b;
        a1.m_data = res;
        return a1;
    };
    cout << c().m_data << endl;

    return 0;
}

```

○ 函数体

和普通函数一样,lambda匿名函数包含的内部代码都放置在函数体中.该函数体内除了可以使用指定传递进来的参数之外,还可以使用指定的外部变量以及全局范围内的所有全局变量。需要注意的是,外部变量会受到以值传递还是以引用传递方式引入的影响,而全局变量则不会.换句话说,在lambda表达式内可以使用任意一个全局变量,必要时还可以直接修改它们的值。

全局变量的值可以修改

```

#include<iostream>
using namespace std;

double d = 5.0;

int main()
{

    auto c = []() /*-> A*/
    {
        double b = 2.0;
        d = 6.0;
        double res = d / b;
        return res;
    };
    cout << c() << endl;
    cout << d << endl;

    return 0;
}

```

- **问题1:** lambda函数中的 = 表示lambda函数的返回值吗

```
auto add = [](int x, int y) { return x + y; };  
int a = add(3, 4);
```

在这个lambda表达式中，其实是创建一个函数对象，它封装了一个可以调用（即执行）的操作。当你调用 `add(3, 4)` 时，你实际上是调用这个函数对象的调用运算符，并且这个调用会返回 7。这个 7 是 `add(3, 4)` 这个函数调用表达式的返回值，而不是 `add` 本身的“返回值”。

为了澄清，这里有两个概念需要区分：

Lambda表达式的返回值：当你调用一个lambda函数对象时，它可能会返回一个值。这个值是由lambda函数体中的返回语句确定的。

Lambda表达式创建的对象：lambda表达式创建一个函数对象，这个对象可以被存储、传递和调用。函数对象本身不是一个返回值，而是一个可调用实体。

• 问题2：c++lambda函数的作用

C++中的lambda函数（也被称为lambda表达式或匿名函数）是一种非常强大的工具，它允许你定义一个可以在需要时立即使用的匿名函数对象。lambda函数的主要作用包括：

代码简洁与可读性：lambda函数允许你在需要的地方直接定义并使用一个小函数，而无需事先声明一个单独的函数或函数对象。这有助于减少代码冗余，并使代码更加紧凑和易于理解。

就地定义函数逻辑：有时，你可能需要在某个特定的作用域或上下文中执行一段逻辑，而这段逻辑只在这个地方使用一次。在这种情况下，使用lambda函数可以就地定义这段逻辑，而无需在全局作用域或类的成员函数中定义它。

算法和容器的配合：C++标准库中的许多算法（如 `std::find_if`、`std::transform` 等）都接受一个函数对象作为参数。lambda函数可以很方便地作为这些算法的参数，用于定义算法的行为。

闭包特性：lambda函数可以捕获其所在作用域中的局部变量，并在其函数体内使用这些变量。这种捕获变量的能力使得lambda函数可以访问和操作外部作用域的变量。

状态封装与传递：lambda函数可以封装状态信息，并将其传递给其他函数或对象。这在需要传递函数逻辑和相关数据时非常有用。

• 问题3：c++lambda函数和普通函数的区别？

C++中的lambda函数和普通函数（非lambda函数）之间存在一些关键的区别。这些区别主要体现在定义方式、作用域、生命周期以及使用场景上。以下是它们之间的主要区别：

1. 定义方式：

普通函数：通常在类的外部或内部定义，具有显式的函数名、返回类型和参数列表。

Lambda函数：是匿名函数，直接在代码中使用lambda表达式定义，没有显式的函数名，但可以有返回类型和参数列表。

2. 作用域和捕获：

普通函数：其作用域通常跨越整个文件或类，并可以通过其名称在代码中多次调用。普通函数不直接捕获外部变量。

Lambda函数：具有局部作用域，只在定义它的代码块内可见。它们可以捕获其所在作用域中的局部变量，使得这些变量在lambda函数体内可用。捕获可以是按值或按引用。

3. 生命周期：

普通函数：它们的生命周期通常与程序或类实例的生命周期相同。

Lambda函数：它们的生命周期取决于它们被创建和使用的上下文。如果lambda函数被赋值给一个对象或作为参数传递给其他函数，则其生命周期可能会与这些对象的生命周期相关联。

4. 语法简洁性：

普通函数：需要显式声明函数名和返回类型，语法上可能较为繁琐。

Lambda函数：可以就地定义，无需显式函数名，语法简洁，特别适用于需要一次性使用的函数逻辑。

5. 使用场景：

普通函数：适用于需要多次调用和跨作用域使用的函数逻辑。

Lambda函数：特别适用于需要就地定义函数逻辑的场景，如算法库中的回调函数、排序比较函数、事件处理等。它们使得代码更加紧凑和模块化。

6. 性能：

在性能方面，普通函数和lambda函数之间没有本质的区别。它们都被编译器转换为机器代码并执行。然而，由于lambda函数可能涉及捕获外部变量，这可能会增加一些内存开销。